

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.2a: Transformer Overview and the Self-Attention Mechanism

Zhigang Feng

Spring 2026

Topic 7.2a Agenda

1. **Transformer High-Level** — encoder/decoder, Vaswani 2017 motivation
2. **Self-Attention Deep Dive** — Q/K/V, multi-head, positional encoding (sinusoidal → RoPE/ALiBi)

Arc: T1a Why+Tokens → T1b Embeddings → T2a Attention → T2b Transformer → T3a Training → T3b Limits → T4 Applications.

Transformer High-Level

How the Transformer Works in Plain Language

- Consider the sentence: *“The Fed raised rates to combat inflation.”*
- When you read the word **rates**, you instinctively ask: *“rates of what? raised by whom? why?”* — your eye jumps back to *Fed* (subject) and forward to *inflation* (motivation).
- **A Transformer does exactly this, for every token simultaneously:**
 1. Every token asks: *“which other tokens are most relevant to understanding me?”*
 2. Each token collects a weighted blend of information from every other token in the sequence
 3. All positions are processed in **parallel** — not one-by-one as in an RNN
- Repeat this “everyone looks at everyone” operation L times (e.g. $L = 96$ in GPT-3) with different learned weights, and the model builds deeply contextualized representations.
- **Engineering name for this operation:** scaled dot-product self-attention. The next slides unpack the five steps in detail.
- **Contrast with Word2Vec / GloVe.** Those models produced *one vector per word type*, independent of context — *bank* in “river bank” and *bank* in “central bank” share the same $\mathbf{u}$. A Transformer produces *one vector per token occurrence*, computed from its neighbors — the two *banks* end up at different points in $\mathbb{R}^d$. **Static** $\rightarrow$ **contextual** is the single conceptual jump.

The Transformer as a Function

- A Transformer is a function

$$\mathcal{T}: (\mathbb{R}^d)^n \longrightarrow (\mathbb{R}^d)^n, \quad [e^{(1)}, \dots, e^{(n)}] \mapsto [e'^{(1)}, \dots, e'^{(n)}]$$

mapping n input vectors to n output vectors of the same dimension. Inputs are token embeddings plus positional encodings; outputs are *contextualized* embeddings.

- **What “enriched” means.** Each $e'^{(i)} \in \mathbb{R}^d$ encodes at once (a) the identity of token $t^{(i)}$, (b) its grammatical/semantic role in *this* sequence, (c) a weighted blend of every other token's contribution (causally masked in autoregressive models).
- **Important:** these are the Transformer's direct output — an *intermediate representation*, **not the final answer**. Inside: L stacked layers, each containing self-attention and a feed-forward network.
- **Turning embeddings into task output.** A small *task head* sits on top:
 - Generation: project $e'^{(n)}$ onto the vocabulary $\rightarrow$ softmax $\rightarrow$ sample a token (Part VII).
 - Classification: project $e'^{([\text{CLS}])}$ onto class labels $\rightarrow$ softmax $\rightarrow$ pick a class.

Positional Encodings: Why They Are Needed

- **Problem:** self-attention is *permutation-equivariant* in its inputs. The tokens $\{\textit{The, Fed, raised, rates}\}$ give the same internal state whether we read “*The Fed raised rates*” or the scrambled “*Rates raised the Fed*” — yet the economic meaning is opposite (agent and patient are reversed). We must inject position.
- **Solution:** add a position vector $p(i)$ to each token embedding before the first layer:

$$e^{(i)} = \mathcal{E}(t^{(i)}) + p(i).$$

- Three families of position encodings dominate practice. The choice matters most for *long contexts* and *length extrapolation*.
- **Take-away:** a Transformer must be *told* where each token is. The mechanism is simple but indispensable.

Sinusoidal, Learned, and Rotary (RoPE)

- **Sinusoidal** (Vaswani et al., 2017) — fixed, no parameters:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d}), \quad \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d}).$$

Multiple frequencies $\Rightarrow$ relative offsets become linear functions; theoretically extrapolates beyond training length.

- **Learned** (BERT, original GPT) — positions $1, \dots, n_{\max}$ each get a learnable vector. Simple; *cannot* extrapolate past $n_{\max}$.
- **Rotary (RoPE)** — modern default in LLaMA, GPT-NeoX, Qwen, Mistral. Apply a position-dependent *rotation matrix* to Q and K :

$$\langle R_{\theta_i} Q_i, R_{\theta_j} K_j \rangle = f(Q_i, K_j, i - j).$$

Position information is baked into attention scores and depends only on *relative* offset $i-j$ — excellent length extrapolation.

- **ALiBi** (a close cousin) adds a linear distance penalty directly to attention scores; very simple, strong long-context behavior.

Position Encodings: Trade-Off Table

Scheme	Params	Where applied	Extrapolates?	Modern use
Sinusoidal	0	input embedding	yes (in theory)	rare today
Learned absolute	$n_{\max} \cdot d$	input embedding	no	BERT, early GPT
RoPE	0	inside Q, K	yes (with scaling)	LLaMA, Qwen, Mistral
ALiBi	H	attention scores	yes	some long-context models

- **Take-away:** a Transformer must be *told* where each token is. Modern LLMs prefer *relative* schemes (RoPE / ALiBi) because they handle 32k+ context windows gracefully.

From Output Embeddings to the Next Token

- For autoregressive generation we focus on the *last* output vector $e'^{(n)}$.
- Project it onto the vocabulary with a linear layer:

$$\text{logits} = W_{\text{vocab}} e'^{(n)} + b_{\text{vocab}}, \quad W_{\text{vocab}} \in \mathbb{R}^{M \times d}.$$

- Convert logits to probabilities with softmax:

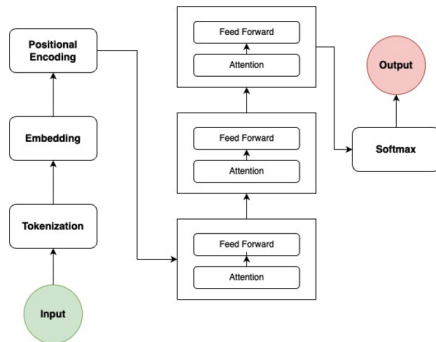
$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \frac{\exp(\text{logits}_w)}{\sum_j \exp(\text{logits}_j)}.$$

- **Sampling strategies:** greedy, top- k , top- p (nucleus), temperature scaling. (More on this in Part VII.)
- Append the chosen token to the context and repeat. **That** is how an LLM “generates text”.

Inside a Layer (Preview)

Each Transformer layer has four steps, each with a distinct job:

- **Self-attention** — *gathers context*: every token asks “who else in this sentence tells me what I mean?” and collects a weighted blend of their values.
- **Residual + LayerNorm** — *safety net*: the original vector is added back (so attention can only *add* information, never destroy it) and then normalized so deep stacks don’t blow up.
- **Feed-forward (MLP)** — *per-token transformation*: each token’s vector is passed through a two-layer non-linear network; this is where most of the model’s memorized knowledge lives.
- **Residual + LayerNorm** again — the same safety net after the MLP step.



Stack L such layers; the same shape $(\mathbb{R}^d)^n$ flows through each one.

Real LLM Scale

Quantity	Symbol	GPT-2	GPT-3	Modern (2024–26)
Layers	L	12–48	96	60–120
Hidden dim	d	768–1600	12288	4096–16384
Context length	n	1024	2048	32k–1M
Vocabulary	M	50k	50k	50k–256k
Parameters	—	0.1–1.5B	175B	70B–>1T

- **Modern open-weight examples (2024–2026):** LLaMA-3 (8B, 70B, 405B), DeepSeek-V3 (671B MoE), Qwen-2.5, Mistral.
- **Modern closed examples:** GPT-4/4o, Claude 3/4, Gemini 1.5/2 (sizes undisclosed).
- All share the same basic architecture; differences are scale, training data, and tuning.

Self-Attention Deep Dive

Self-Attention: The Key Idea

- For each position i in the sequence, ask: “*which other positions are most relevant to position i ?*”
- Compute a set of weights α_{ij} (with $\sum_j \alpha_{ij} = 1$) and form a new representation as a *weighted sum* of all positions.
- **Crucial properties:**
 - Every token can attend to every other token in one step (no recurrence)
 - Weights are computed dynamically from the input, not fixed
 - Operation is parallelizable across positions $\Rightarrow$ trains efficiently on GPUs
- This single operation replaces a tower of recurrent steps and is the central piece that made the Transformer work.

Step 1 — Project to Q, K, V

- For each input embedding $e^{(i)} \in \mathbb{R}^d$, compute three projections:

$$Q_i = W_Q e^{(i)} \quad (\text{query})$$

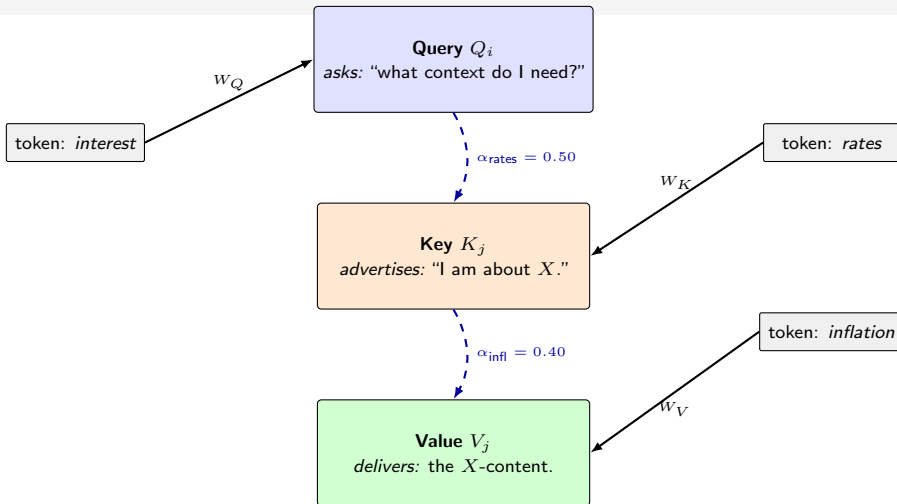
$$K_i = W_K e^{(i)} \quad (\text{key})$$

$$V_i = W_V e^{(i)} \quad (\text{value})$$

where $W_Q, W_K, W_V \in \mathbb{R}^{k \times d}$ are learned matrices and typically $k \leq d$.

- Mnemonic** (information retrieval):
 - Q_i is the *question* that token i asks: “what context do I need?”
 - K_j is the *description* that token j advertises: “here’s what I am about”
 - V_j is the *actual content* that j contributes if matched
- What W_Q, W_K, W_V are trained to achieve:** shape the projections so that the dot product $\langle Q_i, K_j \rangle$ is high when V_j ’s content is useful to what the task needs at position i , and low otherwise. The model discovers which pairings matter from the loss; it is never told in advance.
- Why three separate projections?** Separating roles enables *asymmetric* attention: *rates* can ask a question (Q) that matches *inflation*’s advertisement (K) without *inflation* having to ask the same question. The next slide shows what breaks if we drop or tie any of (Q, K, V) .

Q/K/V at a Glance — Three Roles, One Token



Three roles, kept separate. *Query* (Q_i) is the token's request; *Key* (K_j) is each candidate's advertisement; *Value* (V_j) is what each candidate delivers if matched. Asymmetry matters: *interest* can query *rates* without *rates* querying back. The matrices W_Q, W_K, W_V learn to make these projections shape attention to the loss — nobody hand-codes "attend 13/22

Step 2 — Scaled Dot-Product Scores

- For each pair (i, j) , measure the compatibility between query Q_i and key K_j by their dot product, scaled:

$$s_j^{(i)} = \frac{\langle Q_i, K_j \rangle}{\sqrt{k}}.$$

- **Why divide by $\sqrt{k}$?** As the dimension k grows, dot products tend to grow too, pushing softmax into saturation (one term dominates, gradients vanish). Scaling by $\sqrt{k}$ keeps the variance of $s_j^{(i)}$ around 1.
- **Search-engine analogy:**
 - Q_i is your search query
 - $\{K_j\}$ are document tags / metadata
 - Scores $s_j^{(i)}$ rank tags by relevance to the query
 - Top-ranked tags will let us read the corresponding “documents” V_j

Step 3 — Softmax → Attention Weights

- Convert scores into a probability distribution over positions:

$$\alpha_j^{(i)} = \frac{\exp(s_j^{(i)})}{\sum_{r=1}^n \exp(s_r^{(i)})}.$$

- Properties:
 - $\alpha_j^{(i)} \geq 0$ and $\sum_j \alpha_j^{(i)} = 1$
 - Concentrated weights $\Rightarrow$ “hard” attention to a few positions
 - Diffuse weights $\Rightarrow$ “soft” attention spread across many
- **Causal masking** (autoregressive models): when predicting position $i + 1$, set $s_j^{(i)} = -\infty$ for $j > i$ before the softmax. This prevents the model from peeking at the future during training.
- **Encoder models** (BERT) use bidirectional attention: every token sees every other.
- **Why softmax?** We want the output to be a *convex combination* of value vectors — weights that sum to 1, all non-negative. Softmax converts any real-valued score vector into exactly such a probability distribution, enabling an interpretable weighted average of content across the sequence.

Step 4 — Weighted Sum of Values

- Given the attention weights $\alpha_j^{(i)}$, the attention output for position i is the weighted sum of all value vectors:

$$\text{AttnVec}_i = \sum_{j=1}^n \alpha_j^{(i)} V_j \in \mathbb{R}^k.$$

- Interpretation:** token i “reads” content from all positions in the sequence, but pays most attention to those whose keys best match its query.
- Long-range, short-range, and same-position interactions all happen in the same step — contrast with RNNs where information has to propagate through many time steps.
- Note: V_j is what is being passed around. K_j only decides *how much* to pass; the content delivered is V_j .
- Why a weighted sum rather than a hard lookup?** Instead of selecting a single “most similar” token, the model retrieves a *blend*: for *interest* in a monetary-policy sentence, it mixes V_{rates} , $V_{\text{inflation}}$, and V_{raised} into one vector that reflects all three contexts simultaneously — richer than any single lookup could provide.

Step 5 — Project Back to Dimension d

- The attention output lives in $\mathbb{R}^k$, but we need a vector in $\mathbb{R}^d$ to feed the next layer. Project back with a learned matrix $W_O \in \mathbb{R}^{d \times k}$:

$$\tilde{e}^{(i)} = W_O \text{AttnVec}_i.$$

- This gives the contextualized representation of token i *after* self-attention.
- **Why project back to $\mathbb{R}^d$?** In multi-head attention each head operates in a smaller subspace $\mathbb{R}^k$ (with $k = d/H$) to keep compute manageable. Projecting back with W_O after concatenating heads restores the common dimension d , so the result flows into the next layer without any shape change.
- **Five-step recap:**
 1. Project: Q_i, K_i, V_i
 2. Score: $s_j^{(i)} = \langle Q_i, K_j \rangle / \sqrt{k}$
 3. Softmax: $\alpha_j^{(i)}$
 4. Weighted sum: $\text{AttnVec}_i = \sum_j \alpha_j^{(i)} V_j$
 5. Output projection: $\tilde{e}^{(i)} = W_O \text{AttnVec}_i$
- All matrices (W_Q, W_K, W_V, W_O) are learned by backpropagation from a task loss.

Worked Example — Part 1

Sentence: “The central bank raised interest rates amid rising inflation.”

Tokenization (9 word tokens, numbered left to right):

The central bank raised interest rates amid rising inflation
1 2 3 4 5 6 7 8 9

Goal: produce $e'^{(5)}$, the contextualized vector for *interest* — one that encodes how the word is used *in this sentence* (the interest of monetary-policy rates, not “curiosity” or “financial stake”).

- **Attention score** $s_j^{(5)} = \langle Q_5, K_j \rangle / \sqrt{k}$ measures how much *interest* should listen to token j . Suppose, after scaling, the model finds:

Token j	$s_j^{(5)}$
rates ($j = 6$)	2.3
inflation ($j = 9$)	1.9
raised ($j = 4$)	1.1
bank ($j = 3$)	0.5
the ($j = 1$)	-0.2

Intuitively: *interest* is most strongly tied to *rates* and *inflation* — the words that fix its meaning here.

Worked Example — Part 2

- **Softmax** $\rightarrow$ **attention weights**. $\alpha_j^{(5)}$ is the *fraction* of attention *interest* pays to token j — a probability, with $\sum_j \alpha_j^{(5)} = 1$:

$$\alpha_6^{(5)} \approx 0.50, \quad \alpha_9^{(5)} \approx 0.40, \quad \alpha_4^{(5)} \approx 0.05, \quad \dots$$

50 % of the attention goes to *rates*, 40 % to *inflation*, a sliver to everything else.

- **Weighted sum of value vectors** (what the token actually “reads”):

$$\text{AttnVec}_5 \approx 0.50 V_6 + 0.40 V_9 + 0.05 V_4 + \dots$$

- **Project back:** $\tilde{e}^{(5)} = W_O \text{AttnVec}_5$.
- **Result:** $\tilde{e}^{(5)}$ is a vector in $\mathbb{R}^d$ that represents “*interest, in a context dominated by rates and inflation*” — exactly the meaning we want for monetary-policy text.
- The same computation runs in parallel for every position i , producing $\tilde{e}^{(1)}, \dots, \tilde{e}^{(9)}$ — every token gets its own context-aware vector in one step.

How Q , K , V Are Learned

- **What is trainable (the “controls” θ):** in every layer, W_Q, W_K, W_V, W_O , plus the FFN weights W_1, W_2, b_1, b_2 , LayerNorm parameters, the token embedding matrix $\mathcal{E}$, and the output head W_{vocab} . All are initialized randomly and updated by gradient descent on a **single global loss**.
- **Notation:** $t^{(i)}$ is the i -th token of a training sequence s . The sum $\sum_s \sum_i$ runs over every position i in every sequence s in the corpus.

- **Common training objectives:**

- **Autoregressive LM** (GPT-style):

$$\mathcal{L}(\theta) = - \sum_s \sum_i \log P_{\theta}(t_s^{(i+1)} \mid t_s^{(1:i)}).$$

- **Masked LM** (BERT-style): hide tokens at random and predict them from bidirectional context.
- **Classification fine-tuning:** cross-entropy on a sentence-level label.
- **One loss, all parameters.** Backpropagation sends gradients through softmax, dot products, Q/K/V projections, residuals, LayerNorm, *and the FFN* — one backward pass updates every parameter together. There is no separate training signal for attention vs. the feed-forward block.
- Nothing tells the model “attend to neighbors” or “ignore stopwords” — it discovers attention patterns from data.

Multi-Head Attention

- One (Q, K, V) trio captures only one “view” of the sequence. **Multi-head attention** runs H such trios in parallel and concatenates their outputs.
- Each head h has its own projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, all of dimension $k = d/H$.
- After running H attention computations independently, concatenate and project:

$$\text{MultiHead}_i = W_O^* \left[\text{AttnVec}_i^{(1)}; \dots; \text{AttnVec}_i^{(H)} \right].$$

- **Do heads have different objectives?** *No — all H heads share the same training loss* (e.g. next-token cross-entropy). What differs is their random initialization. Specialization is **emergent**: under one global loss, H parallel feature detectors diversify because identical copies of the same head would be redundant.
- **Observed specializations in financial text:**
 - Head 1: monetary-policy relations (*inflation* $\leftrightarrow$ *interest*)
 - Head 2: temporal phrases (*Q1, next month, by year-end*)
 - Head 3: hawkish vs. dovish stance markers

Nobody told the heads what to look for — they were not assigned separate objectives.

- Total parameter cost is essentially the same as a single-head attention with the full dimension d ^{21 / 22}

T2a Wrap-Up: Attention as a Building Block

- **What we have so far:** a single self-attention block that turns each token into a context-aware vector via Q , K , V projections, scaled scores, softmax, and a weighted sum of values.
- **Three reusable ideas** (returning in every later topic):
 - *Query-key matching* — the same logic powers RAG retrieval (Lec08).
 - *Asymmetric three-projection design* — queries, keys, values stay distinct because their jobs differ; this asymmetry is what makes attention more expressive than a similarity lookup.
 - *Soft, differentiable lookup* — the engine behind LLM “reasoning” over context, including JSON tool calls in Lec09.
- **Next: Topic 7.2b.** One attention block is not a Transformer. We add residuals, LayerNorm, a feed-forward network, and stack L such layers — and meet the Vaswani 2017 encoder/decoder diagram. Then a case study (Kim et al. 2024) applies the same machinery at the *paragraph* level on 10-K filings.